Software Systems

Session 4

Requirements Modeling

Y. Raghu Reddy
Software Engineering Research Center
IIIT Hyderabad, India

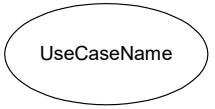
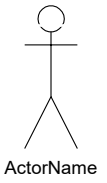
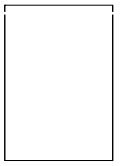
INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
HYDERABAD

Why Model Requirements?

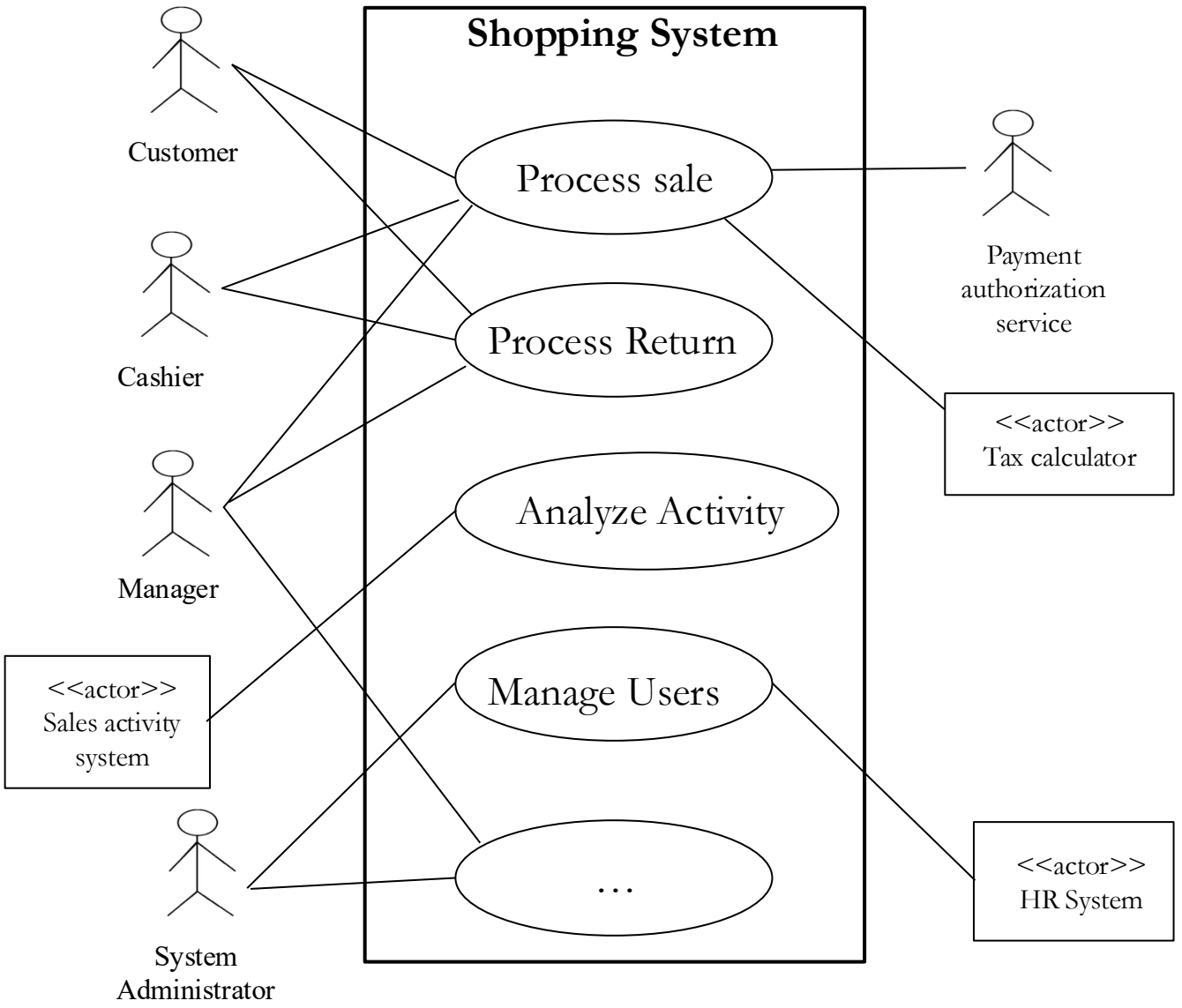
- Converts **textual requirements** into **structured, visual models**
- Helps **validate requirements early** before design begins
- Acts as a **bridge** between *business needs* and *system design*
- Clarifies **system functionality** for all stakeholders
- Most common & popular:
  - Use Case Modeling
  - Conceptual/Domain Modeling

Use case Modeling

- A technique to **capture functional requirements** from a user's perspective
  - *Use cases* emphasize thinking from the viewpoint of the users
  - Shows **how users (actors) interact with the system** to achieve their goals
- Focuses on *what* the system should do, not *how* it's implemented
- It describes an end-to-end process: from when a user starts to use the system for a particular purpose until they are done (for that purpose).

Construct	Description	Syntax
use case	A Sequence of actions, including variants, that a system (or other entity can perform, interacting with actors of the system	
actor	A role played by an entity that interacts with the subject (e.g., system, subsystem, class)	
system/subject boundary	Represents the boundary between the subject and the actors who interact with the subject	

Use Case Diagram - Example



Adapted from Applying UML & Patterns, Craig Larman

Identifying Actors and Use cases

1. **Identify Actors:** External entities that initiate interaction.
2. **Identify Goals:** What does each actor want to achieve?
3. **Define Use cases:** One per goal.

Actor	Goal	Use Case
Customer	Withdraw money	Withdraw Cash
Customer	View account balance	Check Balance
Bank Server	Verify account	Validate Account

Use Cases as Requirements

- Use cases can be used to capture functional requirements
  - System attributes associated with a system operation can be documented in a use case

- Not all requirements can be captured by use cases
  - System attributes that span use cases are documented as supplementary requirements

Use Case Template

Use Case Number	EU-xxxx : Indicates an essential use case, i.e., a use case that describes activity in system independent terms	
Use Case Name	*Enter name of Use Case. (Start with a Verb)*	
Overview	*Describe the purpose of the Use Case and give a brief description.*	
Type	*Enter Use Case priority* (primary, secondary, optional)	
Actors	*List all actors that participate in this Use Case. Indicate the actor that initiates the use case by placing "initiator" in brackets after the actor name. Also, indicate primary actors by placing "primary" in brackets after actor name. This may also be separated as Primary actor, Secondary actor, Stakeholders and Interests.*	
Properties (Special Requirements – if any)	***Performance:***	
	Security:	
	Other:	
Tech/Data Constriants/Variations		

Use Case Template – cont'd

Pre-condition	*Enter the condition that must be true when the main flow is initiated. This should reference the conceptual model.*	
Flow	***Main Flow:*** *Steps should be numbered.*	
		Subflows: *Break down of main flow steps*
		Alternate Flows: *Include the post condition for each alternate flow if different from the main flow.*
Post Condition	*Enter the condition that must be true when the main flow is completed. This should reference the conceptual model. Include the following information in this section:*	
Cross References	*References to other Use Cases or textual requirements that relate to this Use Case.*	

Use Case Instance (Scenario)

- A **scenario** is a specific sequence or path in a use case.
- Each use case has multiple scenarios
  - Main flow: main success scenario
  - Alternative or exception scenarios

- A **use case instance** is an execution of a scenario.
  - Often use case instance and scenario are used synonymously in informal discussions.

Use Case: Withdraw Cash
Scenario 1: Valid card and balance → Cash dispensed
Scenario 2: Invalid PIN → Access denied
Scenario 3: Insufficient balance → Transaction cancelled

Relationship Between Use Case Model, Use Case, and Scenario

Concept	Description	Example
Use Case Model	The complete diagram showing all actors and use cases	ATM System Diagram
Use Case	A single functional goal	Withdraw Cash
Scenario	A specific flow of steps in that use case	Invalid PIN scenario

Thank you !

Levels of Rigor

- **Brief**: One paragraph summaries of functionality
- **Casual**: Multiple paragraphs that cover multiple scenarios
- **Fully-dressed** (Detailed): Structured, detailed description of scenarios

Essential vs. Concrete Use Cases

- **Essential Use Cases** describe functionality in implementation independent terms
  - Requirements level use cases must be essential

- **Concrete Use Cases** describe external functionality in system dependent terms
  - Use cases can be used during design to document externally observable behavior of subsystems

Use Case Modeling Tips

- Use diagrams for presentation purposes only
  - Focus more on text description (Essential use cases)
- Writing essential use cases: Focus on intent
  - Keep user interface terms out
  - Ask "what is the goal?"
- Write "black-box" use cases
  - Do not describe internal operations (e.g., storing to a data base)
- Focus only on interactions between system and actors
  - Ignore interactions between actors
- A use case diagram should
  - contain only use cases at the same level of abstraction
  - include only required actors

How do I know I have a good use case?

- Use case should describe an activity that yields an observable result of value to an actor.

- A use case can describe an elementary business process: a sequence of tasks performed to handle a business event

- Use cases are typically not single steps or single low-level actions.

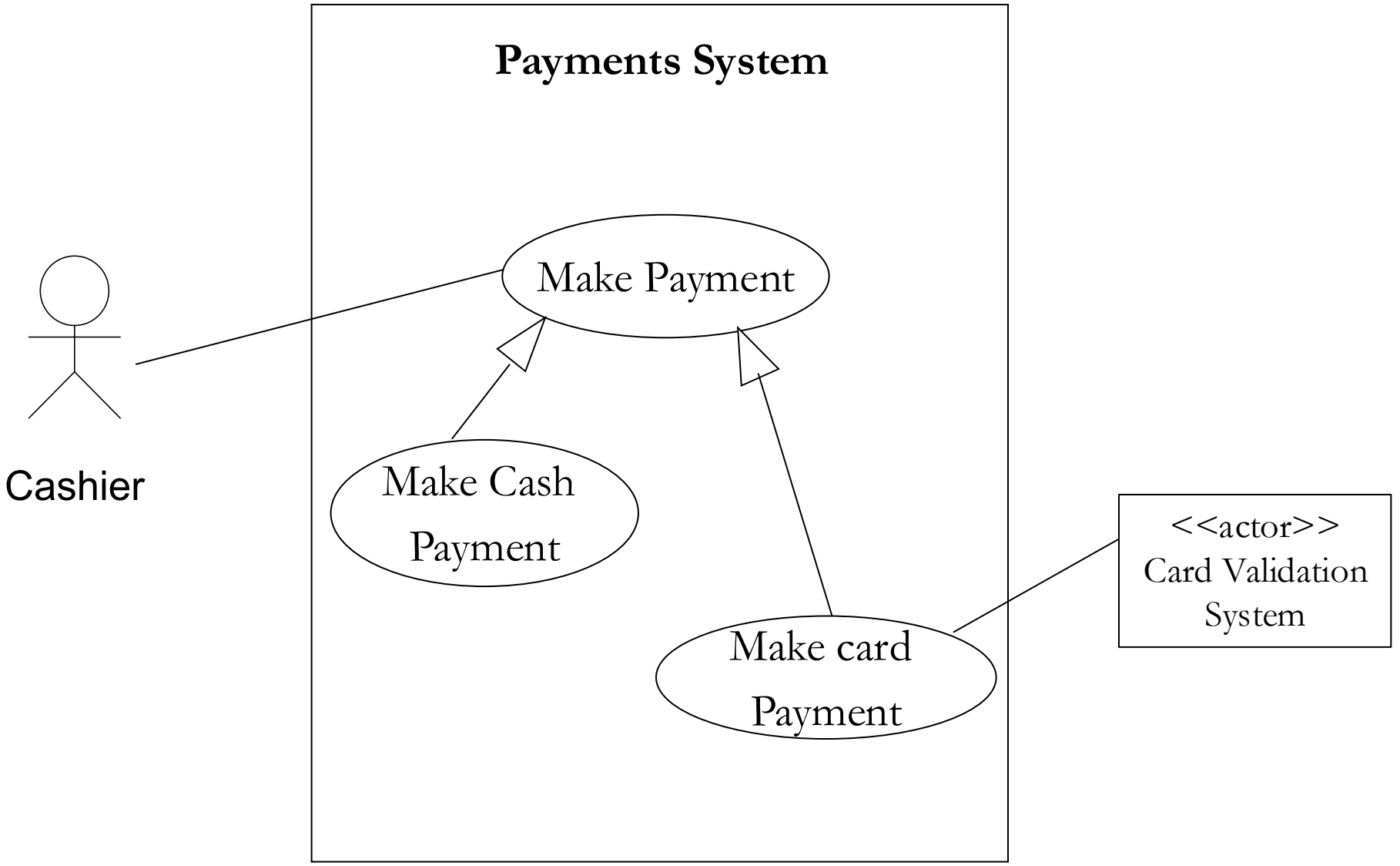
Organizing Use Cases

- Specializing/generalizing use cases
- Including use cases
- Extending use cases

Specializing Use Cases

- Generalizing/Specializing use cases
  - A specialized use case inherits the behavior (sequence of actions) of its parent(s)
  - A specialized use case can override some of the behavior of its parent(s). It can also add to the behavior
  - A specialized use case can be used anywhere the general use case is expected.

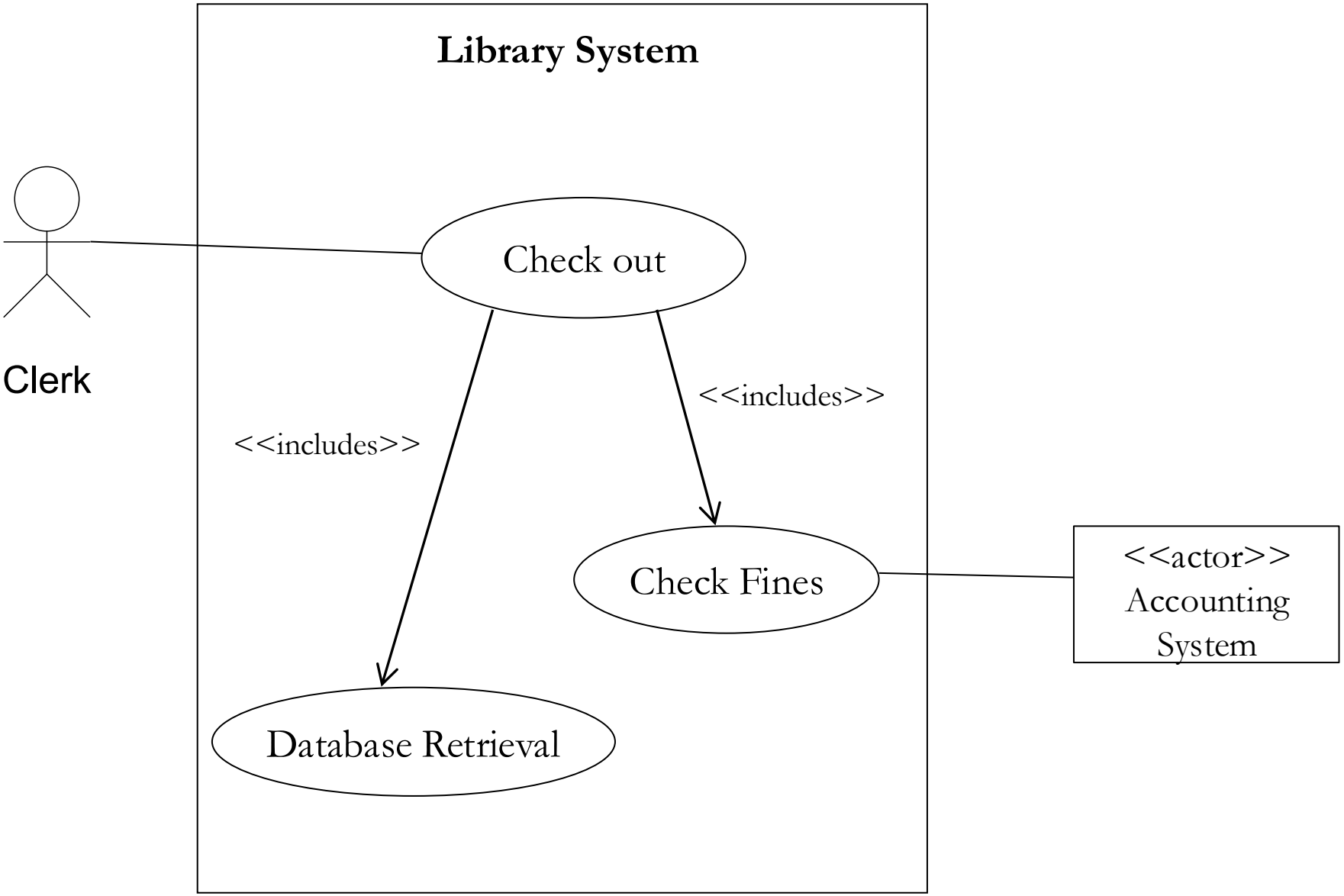
Specializing Use Cases



Including Use Cases

- A use case can include another use case at a specified location
  - Used to avoid writing the same flow of events across a number of use cases.
  - The included use case must not be a stand-alone use case.

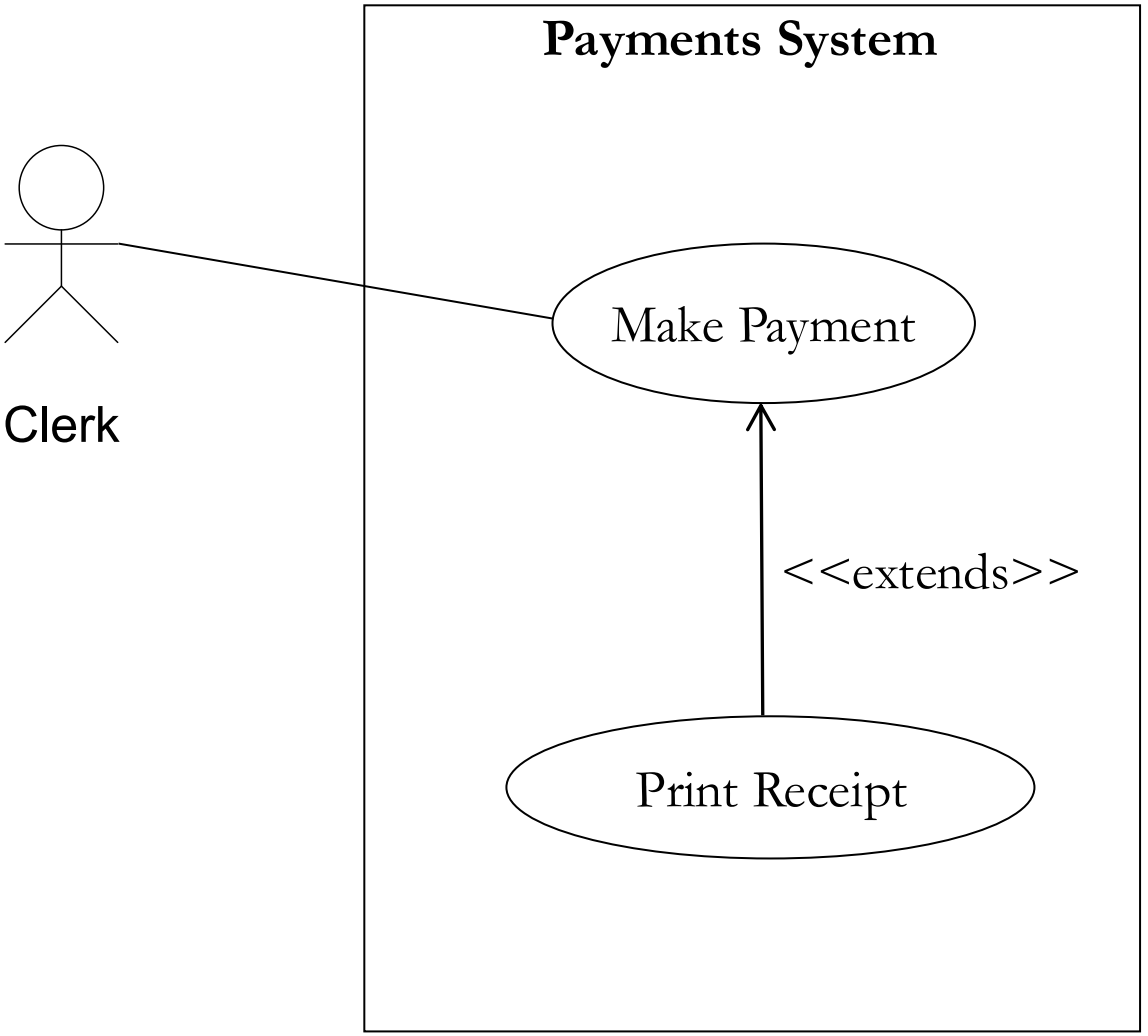
Including Use Cases



Extending Use Cases

- A use case can extend a base use case by incorporating additional behavior at specified locations of the base use case
  - The base use case can act as a stand-alone use case.
  - The base use case can only be extended at specified points called *extension points*.
  - Often used to separate optional behavior from mandatory behavior
  - Also used to model a separate flow that is executed under certain conditions

Extending Use Cases

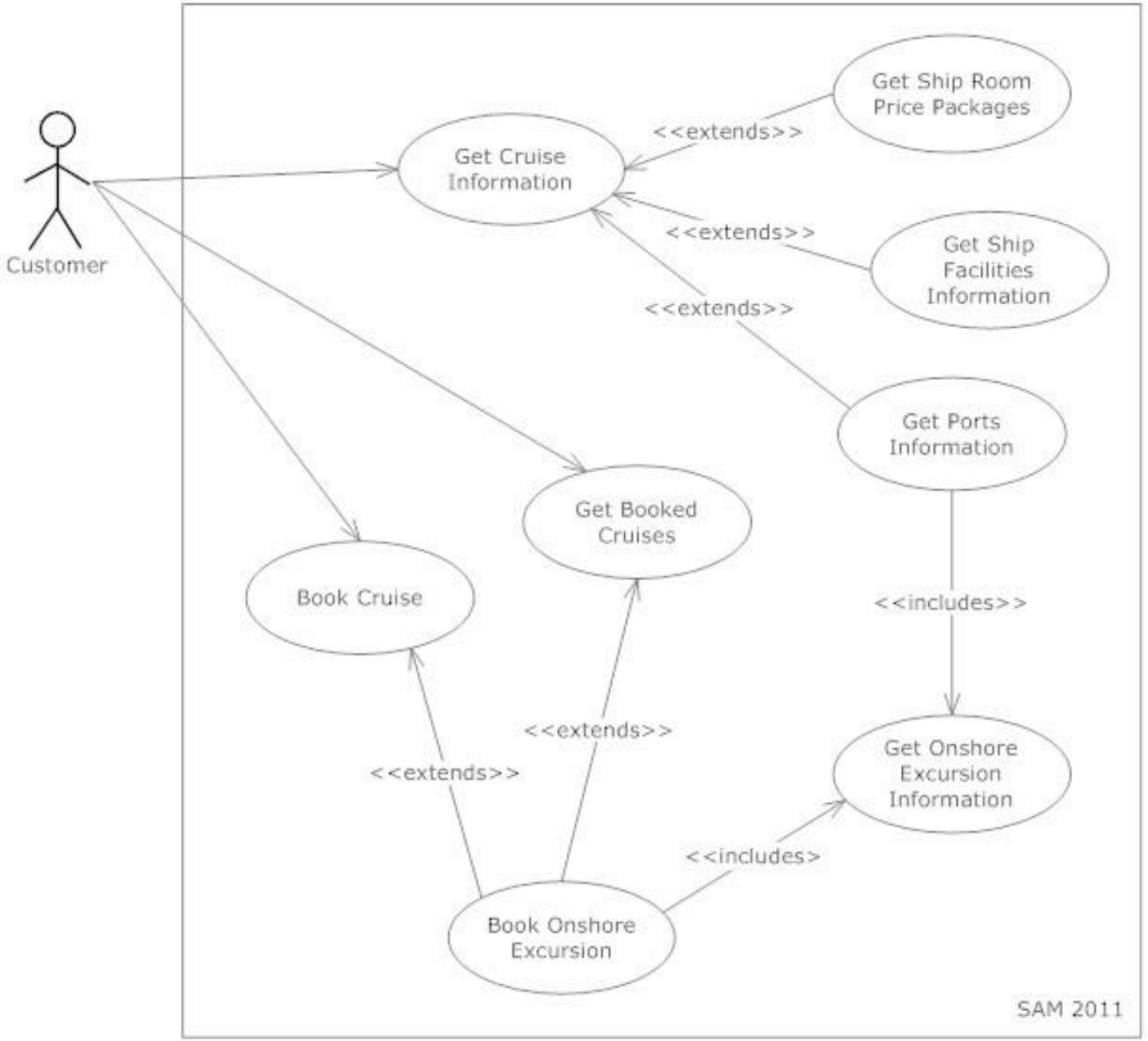


Thank you !

Use case modeling – example problem

The Caribbean Fantasy Tours (CFT) on-line booking system is intended to be a one-stop cruise planning and booking site. Using the site customers will be able to book cruises on any of the shipping lines registered with CFT. A cruise consists of a sequence of Caribbean ports. At a particular port there may be one-or more on-shore excursion packages, where each package is offered by an on-shore tour operator. When booking a cruise, a customer can get information about the ports visited, the ship room price packages, ship facilities, and the on-shore excursions that are available via CFT at each port-of-call.

Example – use case model



Example – Get Cruise Information

Use Case Number:	**EU-0001**
Use Case Name:	Get Cruise Information
Overview:	This use case is used to get information about cruises available for booking. It is the base use case for retrieving information related to a specific cruise. A list of available cruises are displayed with options for retrieving additional information.
Actors:	Customer [initiates, primary]
Precondition:	System displays a list of cruises by name available for booking.
Main Flow:	1. Customer selects option to view one cruise in the list of available cruises. 2. System displays a brief description of the cruise with options to display additional information. 3. <<extension point>> Get Additional Information
Alternate Flow:	3. User selects option to book this cruise.
Alternate Flow:	3. User selects option to get list of available cruises.
Post Condition:	True
Cross References:	Get Ports Information, Get Ship Room Price Packages, Get Ship Facilities Information, Book Cruise

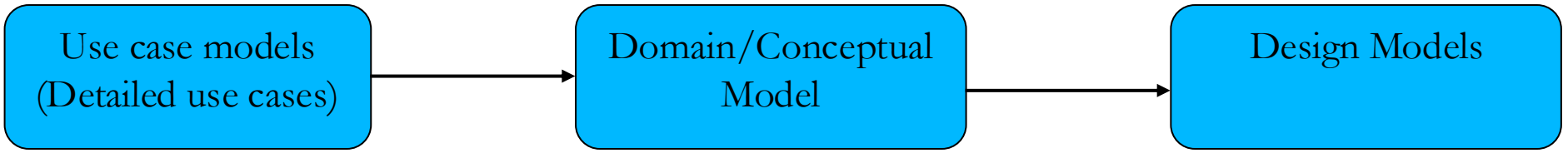
Example – Get Ports Information

Use Case Number:g	**EU-0002**
Use Case Name:	Get Ports Information
Overview:	The purpose of this use case is to retrieve detailed information about the ports that are visited on a cruise. The Customer selects a cruise to display a list of ports. Each port provides a list of onshore excursions.
Actors:	Customer [initiates, primary]
Precondition:	Customer selected option to view cruise information consisting of a sequence of ports.
Main Flow:	1. Customer requests system for ports information. 2. System displays a list of ports by name visited on the cruise with an indicator if it includes an onshore excursion. 3. Customer requests for additional information about a particular port. 4. <<include>> Get Onshore Excursion Information. 5. Customer selects option to go back to cruise overview (EU-0001).
Alternate Flow:	6. Customer selects option to get list of available cruises.
Post Condition:	True
Cross References:	Get Cruise Information, Get Onshore Excursion Information

From Use Cases to Domain/Conceptual Models

- Use cases describe **what happens**
- Domain models describe **what exists**
- Shift from verbs to nouns

Goal: Identify and structure **key concepts** in the system's domain



What Is a Domain Model?

- Structuring of domain concepts
  - Identifies problem concepts and their relationships
- Structural models (for example, UML class diagrams) used to depict structure
- Key Question: What are the concepts of interest in this domain?
  - their attributes?
  - their relationships?
- IMPORTANT: This is a model of problem concepts; these concepts are NOT software objects, but a "visual dictionary" of domain concepts.

Use Case: Withdraw Cash	**Derived Domain Elements**
Actor: Customer inserts card into ATM	**Customer**, **Card**, **ATM**
System: Validates card and PIN	**Card**, **Account**, **Validation** (as process)
Actor: Enters amount to withdraw	**Amount** (attribute), **Transaction**
System: Dispenses cash and updates account	**Cash**, **Transaction**, **Account**
System: Prints receipt	*(Ignore – UI detail)*
Postcondition: Account balance updated	**Account.balance** (attribute)

Let's looks at a Process Sale Use case main flow…

Preconditions: Cashier is identified and authenticated on a sales terminal.

Main flow:

1. Customer arrives at POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sales line item and presents item description, price, and running total. Price calculated from a set of price rules.
   < Cashier repeats steps 3-4 until indicates done >
5. System presents total with taxes calculated.
6. Cashier tells Customer the total, requests payment.
7. Customer pays and System handles payment.
8. System logs completed sale and sends sale and payment information to the external Accounting System and Inventory System.
9. System presents receipt.
10. Customer leaves with receipt and goods (if any).

Guidelines for Domain Modeling

- Extract nouns from use cases
- Identify attributes and relationships
- Validate for completeness and realism

Moving from Use cases to Conceptual model

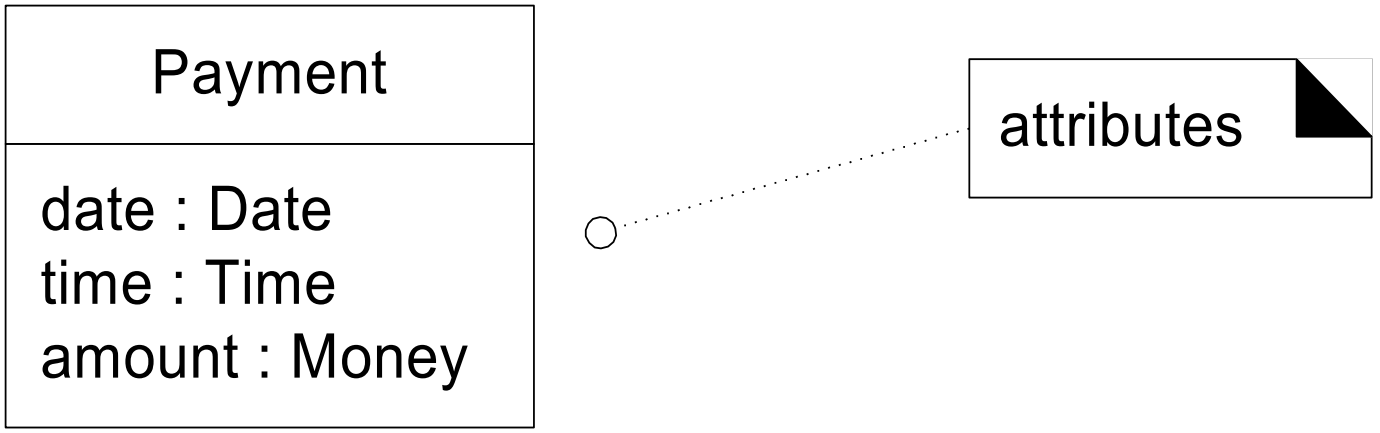
1. ***Customer*** arrives at ***POS checkout*** with ***goods*** and/or ***services*** to purchase.
2. ***Cashier*** starts a new ***sale***.
3. Cashier enters ***item identifier***.
4. System records ***sales line item*** and presents ***item description***, ***price***, and ***running total***. Price calculated from a set of price rules.
   < Cashier repeats steps 3-4 until indicates done >
5. System presents total with *taxes* calculated.
6. Cashier tells Customer the total, requests ***payment***.
7. Customer pays and System handles payment.
8. System logs completed ***sale*** and sends sale and payment information to the external ***Accounting*** System and ***Inventory*** System.
9. System presents ***receipt***.
10. Customer leaves with receipt and goods (if any).

Conceptual Classes

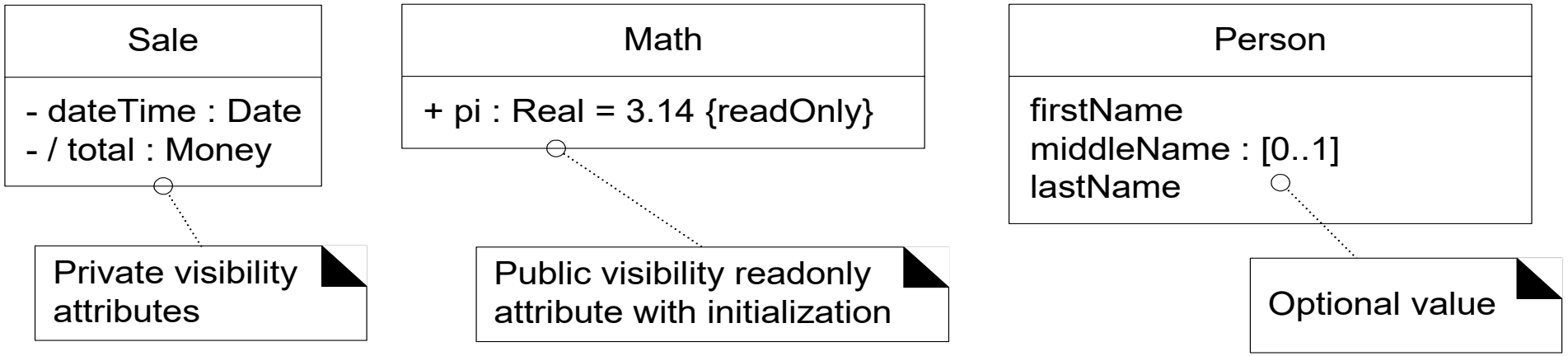
Note: These are not software classses

Attributes

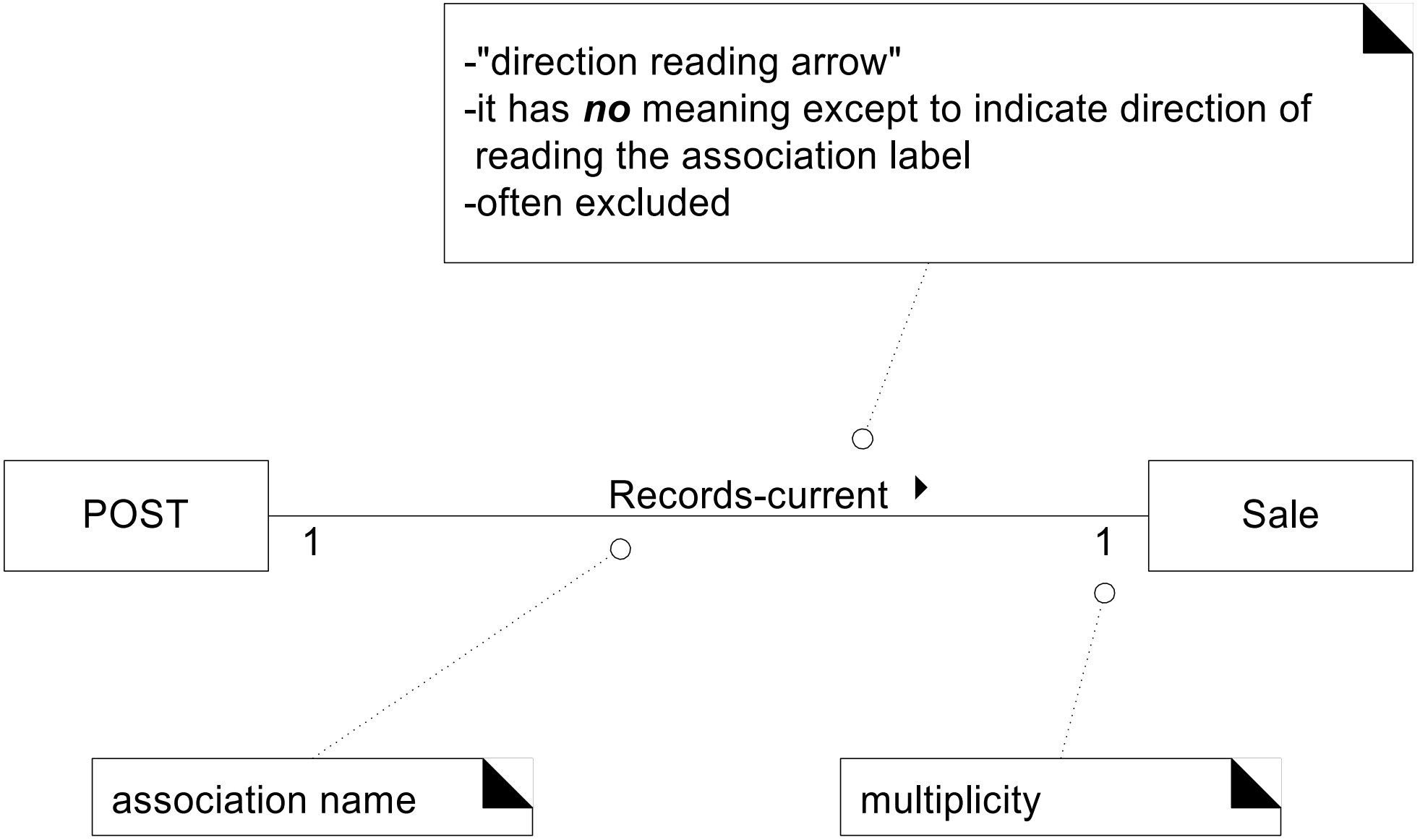
- Show only “simple” relatively primitive types as attributes.
- Connections to other concepts are to be represented as associations, not attributes.



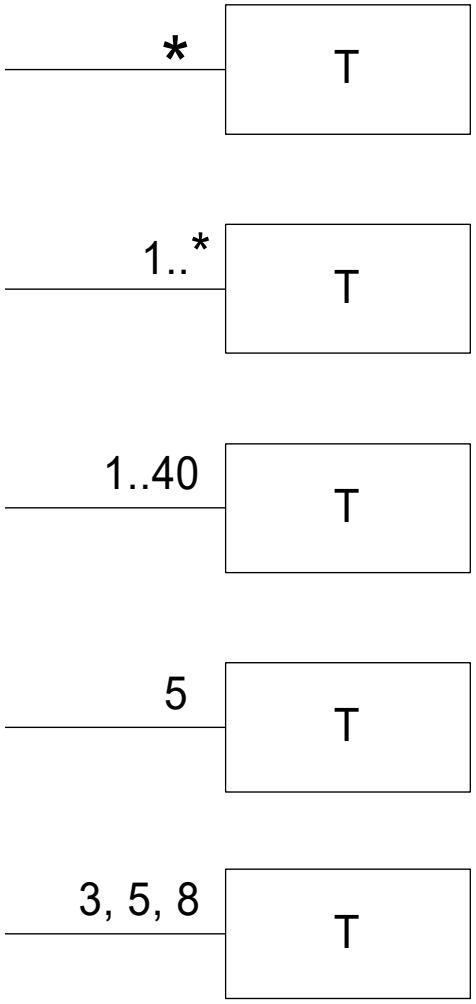
Access Modifiers

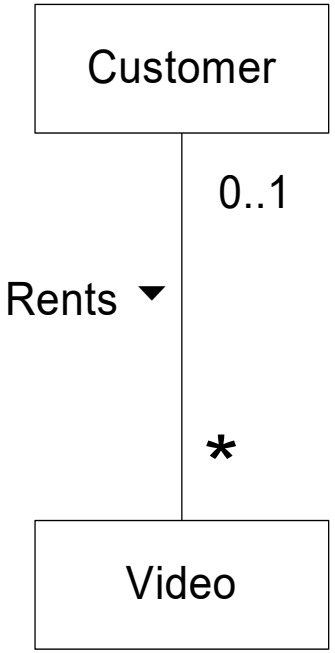


Associations



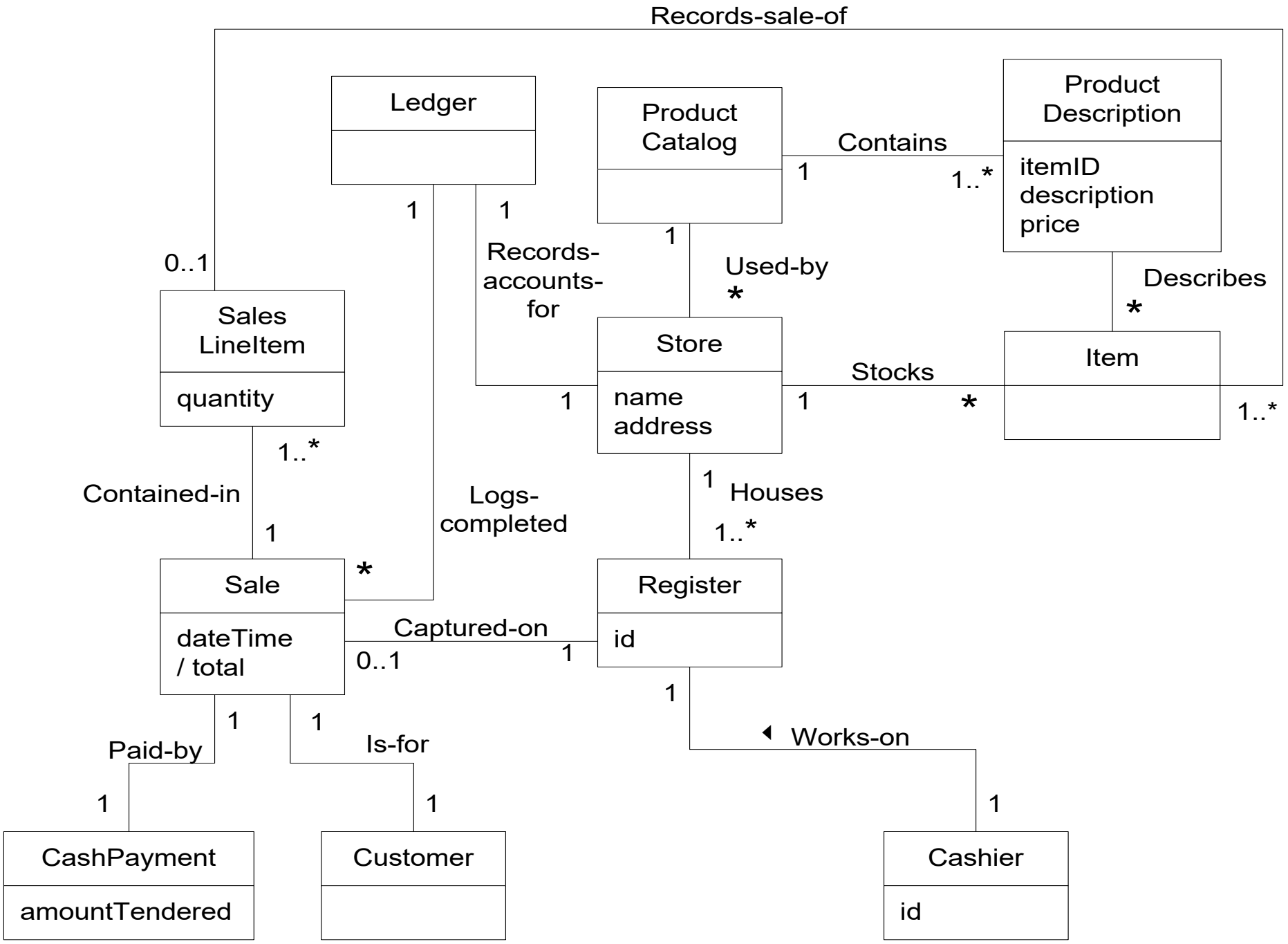
Multiplicity





One instance of a Customer may be renting zero or more Videos.

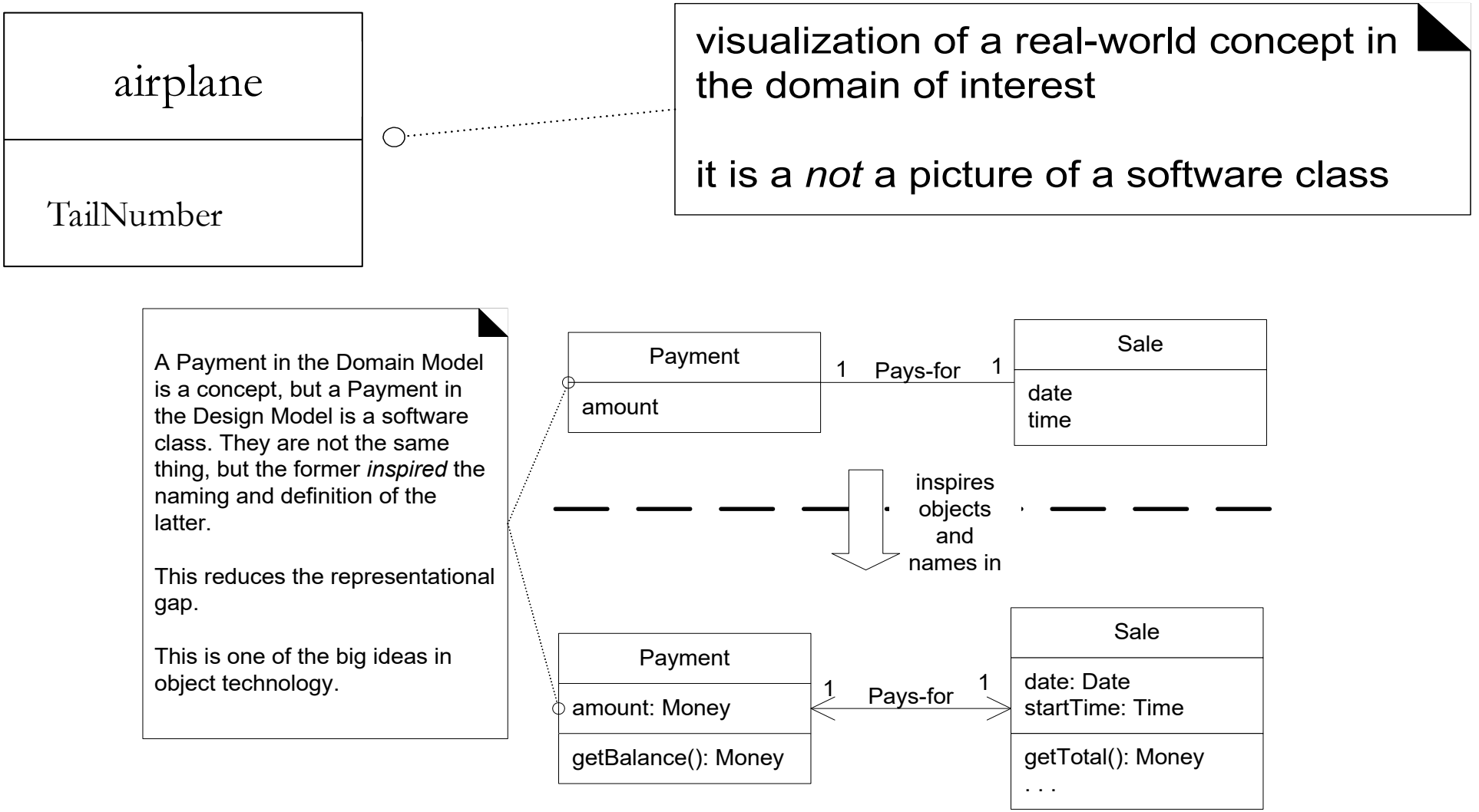
One instance of a Video may be being rented by zero or one Customers.


Records-sale-of
Ledger
Product Catalog
Contains
Product Description
itemID
description
price
1
1..*
1
1
1
Records-accounts-for
Used-by
*
Describes
*
0..1
Sales LineItem
quantity
Store
name
address
Item
Stocks
1
1
*
1..*
1..*
Contained-in
1
Logs-completed
1
Houses
1..*
Sale
dateTime
/ total
*
Register
id
Captured-on
0..1
1
1
1
1
Paid-by
Is-for
Works-on
1
1
1
CashPayment
amountTendered
Customer
Cashier
id

Common pitfalls and corrections

Mistake	Example	Fix
Mixing design elements	Added “Database Connector” class	Remove; stay conceptual
Wrong multiplicity	Transaction as many-to-many	Use appropriate multiplicy
Missing attributes	Transaction had no amount	Add key fields
Vague naming	“Info”, “DataObj”	Use meaningful names

Representing Problem/Domain Concepts



Summary

- Use Case Diagrams and Templates
- Domain/Conceptual Modeling

Next: System Design Models – Class & Sequence Diagrams.

Thank you !